



- GAME DESIGN DOCUMENT -

STARFALL PROTOCOL

MARIA HUHNSTOCK

Table of contents

1. Pitch	1
2. Target Audience	1
3. Design Pillars	1
4. Core Loop	2
5. Mechanics	2
5.1 Player	2
5.2 Enemies (3 types).....	2
5.3 Earth	2
5.4 Boss.....	3
5.5 Upgrade System	3
5.6 Difficulty Scaling	3
5.7 Score	3
5.8 Playtesting & Balancing	4
6. UI / UX.....	4
7. Art Style.....	4
8. Technical Structure	5
System Architecture.....	5
Game State Flow	6
Boss Phases.....	6
Upgrade Selection Flow.....	6
Object Pooling Flow.....	6
9. Scope	7
10. Possible Extensions	7

Starfall Protocol — Game Design Document

Genre: Endless Vertical Shooter **Engine:** Unity 6.6 (URP), C#

1. Pitch

Earth is under attack, and you're its last line of defense: you pilot a small ship through a scrolling starfield and fight through waves of enemies that get tougher over time. Along the way you collect upgrades, grow your score, and try to beat your own high score. Easy to learn, quick to restart.

At the core of the project is clean gameplay architecture: object pooling for stable performance with a full screen of activity, ScriptableObjects to separate content from code, and an upgrade system that stays extensible without becoming unwieldy. All of the art - ship, enemies, boss, UI, Earth - is my own.

2. Target Audience

- Players who enjoy short, replayable arcade games - fans of classic shoot-'em-ups will feel right at home.
- Anyone reviewing my portfolio who wants to see how I structure gameplay code, not just whether the game runs.
- Casual players looking for a quick five-minute session rather than a long-term commitment.

3. Design Pillars

- Instantly readable — enemy types, shots, and upgrades are recognizable at a glance, and the threat to Earth makes the goal immediately tangible. No tutorial needed beyond the first ten seconds.
- Constant pressure — difficulty ramps up continuously, never in sudden jumps. Meant to feel intense, not unfair.
- Upgrades with real weight — every choice is a trade-off
- Precise controls — movement and shooting feel immediate. No lag, no ambiguity about whether a hit landed.

4. Core Loop

1. You start at the bottom of the screen with the base weapon.
2. Enemies come in from above — more of them, and tougher variants, over time.
3. You destroy enemies and collect points.
4. On reaching a score threshold, the run briefly pauses and you pick one of 3 upgrades.
5. In the background, difficulty keeps climbing the whole time.
6. Eventually the boss appears as the run's climax.
7. Either your ship or the Earth beneath you is destroyed → the Game Over screen shows your score, compares it to your high score, and you can restart right away.

5. Mechanics

5.1 Player

- Moves left/right, constrained to the screen area.
- Fires automatically from a central firing point — you focus on positioning and dodging.
- Healthbar in the top-right corner; reduced by collisions with enemies and boss projectile hits.

5.2 Enemies (3 types)

- Drifter - slow, simple movement pattern, low threat.
- Weaver - sinusoidal, evasive movement pattern. Harder to hit, medium threat.
- Rusher - fast, direct approach. Punishes hesitation.

All enemies run on ScriptableObjects (EnemyData) and share an IMovementPattern interface, so a new enemy type later mostly just needs a new data asset instead of an entirely new movement script.

5.3 Earth

- Visible at the bottom of the screen as a small strip, with its own healthbar.
- Enemies that aren't destroyed in time don't just disappear - they collide with Earth and deal damage.
- Damage per enemy type is tied to its speed - faster enemies are harder to intercept and deal correspondingly more damage if they get through.
- If Earth's healthbar hits zero, it's Game Over.

5.4 Boss

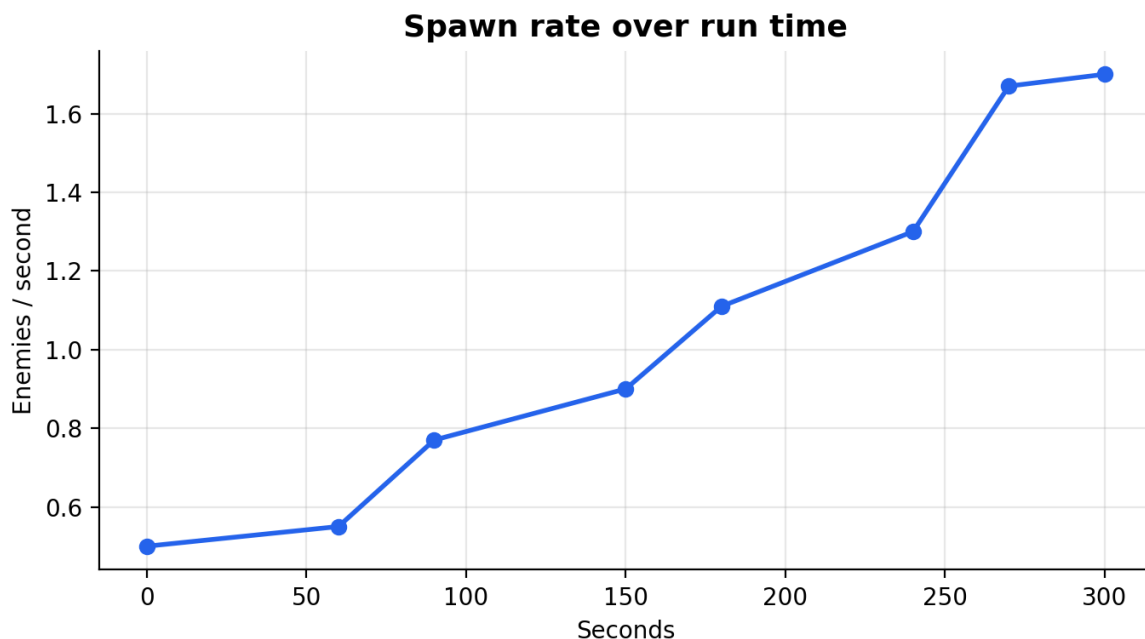
- Appears once per run as the climax.
- Two attack phases, each with its own targeted projectile pattern.
- Serves as the run's central skill check.

5.5 Upgrade System

- 4 upgrades total, offered as a choice of 3 at score thresholds.
- Built on a shared `IUpgradeEffect` interface, driven by `UpgradeData` `ScriptableObjects`, so new upgrades can be added without touching selection or application logic.
- The score threshold to the next upgrade grows after each pick, so the four upgrades spread out across the run instead of clustering early and running dry later.
- Upgrades: Fire Rate, Double Shot (activates the two prepared side firing points), Shield, Speed.

5.6 Difficulty Scaling

Spawn rate and enemy composition scale continuously with survival time - no hard level boundaries, just a steady increase driven by the spawner.



Spawn rate rises continuously over the run's duration, with a noticeable jump after the boss fight.

5.7 Score

- Points increase per enemy destroyed.
- High score is saved locally (`PlayerPrefs`) and shown on the Game Over screen.

5.8 Playtesting & Balancing

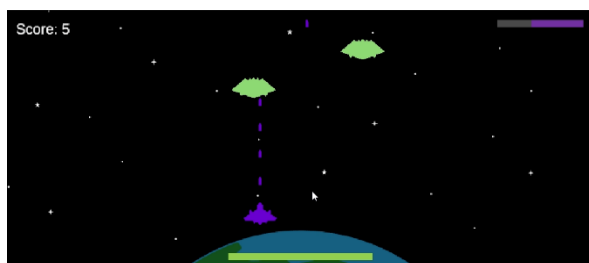
The values for enemies, weapon, and upgrades weren't set once and left alone - they were iteratively tested and adjusted, for instance when upgrades turned out too strong in early test runs and the fire-rate and speed bonuses were scaled back accordingly. The score threshold for upgrades was also changed from a fixed interval to a growing gap, so upgrades spread out across the whole run.

6. UI / UX

- Main menu - Start, optional Settings, Quit.
- HUD - player healthbar top-right, Earth healthbar on the Earth strip at the bottom, current score, high score for reference.
- Upgrade selection screen - appears at thresholds, shows 3 cards, pauses gameplay during selection.
- Pause menu - Resume, Restart, Back to menu.
- Game Over screen - final score, high-score comparison, restart button.

7. Art Style

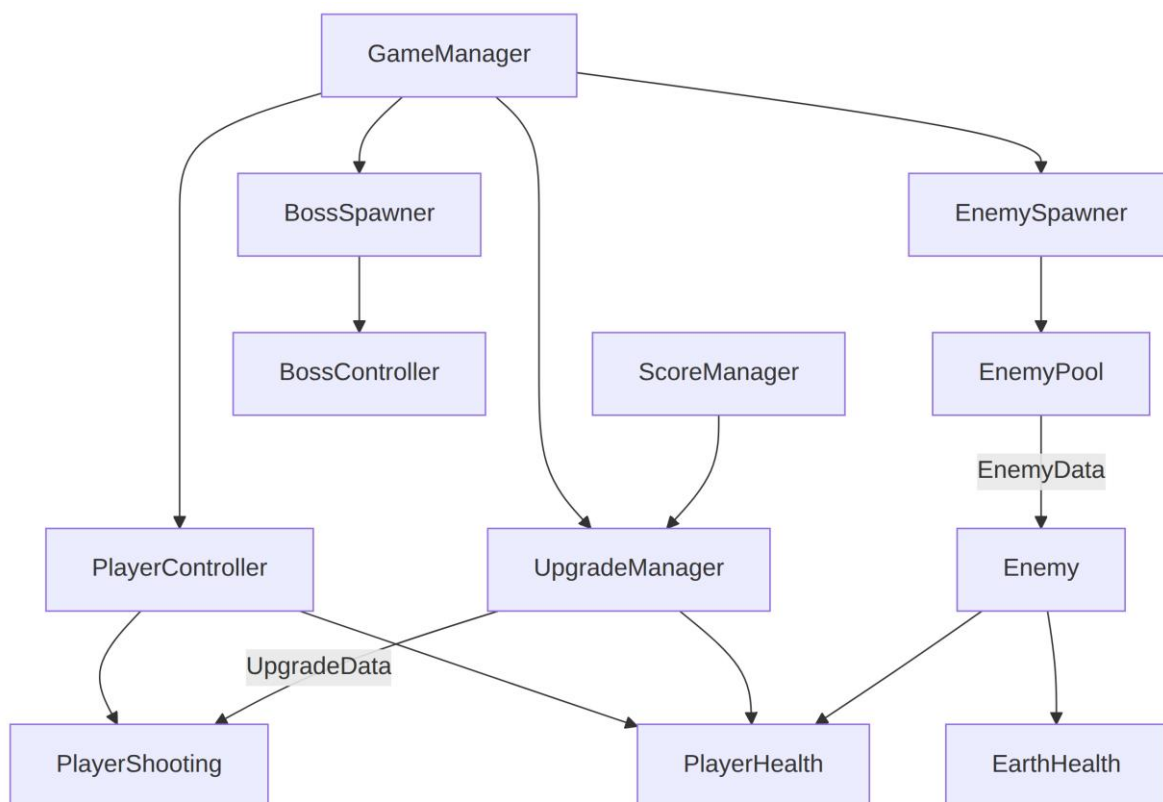
- Every visual element — ship, enemies, boss, UI, and Earth — was designed by me: modeled from geometric shapes in PowerPoint, grouped, and exported as graphics.
- High-contrast silhouettes against a scrolling starfield keep everything readable even with a lot going on.
- Colors aren't purely decorative — they make it easier to tell enemy types apart.
- The UI stays minimal so it doesn't compete with gameplay readability.



8. Technical Structure

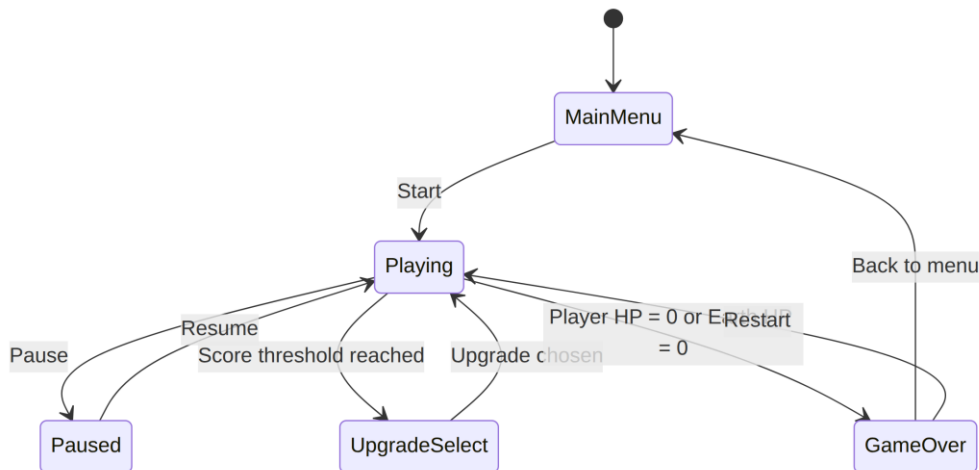
- Object Pooling — a generic `ObjectPool`, plus dedicated pools for projectiles (`ProjectilePool`) and enemies (`EnemyPool`), to avoid instantiation costs during intense waves.
- Data-driven design — `WeaponData`, `EnemyData`, `UpgradeData`, `BossData` as `ScriptableObject`s, keeping design content separate from code logic.
- Interfaces — `IDamageable` for anything that can take damage (player, Earth, enemies, boss), `IMovementPattern` for enemy movement, `IUpgradeEffect` for upgrade behavior.
- Collision logic — dedicated components per interaction type (enemy↔player, enemy↔Earth) instead of centralized damage dispatch, keeping responsibilities cleanly separated.
- Systems — `PlayerController`, `PlayerShooting`, `PlayerHealth`, `EarthHealth`, `EnemySpawner`, `BossSpawner`, `ScoreManager`, `UpgradeManager`, `GameManager` for pause/game-over/restart flow.

System Architecture



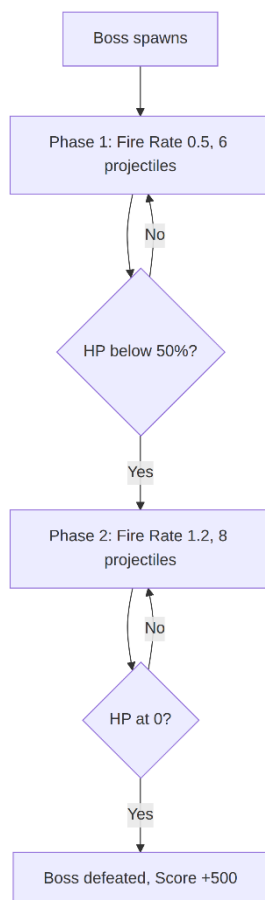
The GameManager orchestrates the subsystems; content flows in exclusively through ScriptableObjects (EnemyData, UpgradeData, BossData), never hardcoded.

Game State Flow



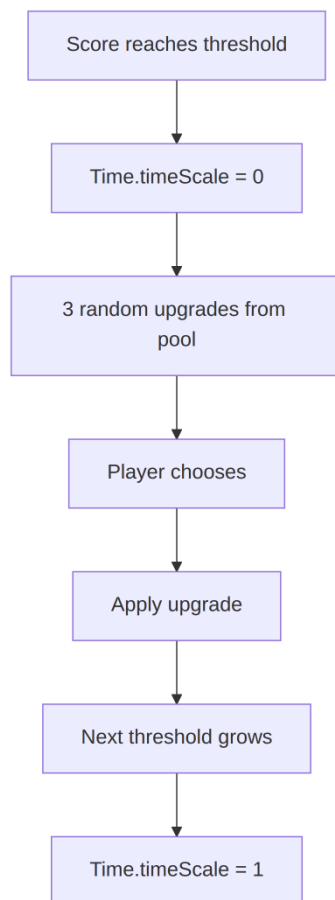
The GameManager centrally drives this state transition, including the time-stop during upgrade selection
(*Time.timeScale = 0*).

Boss Phases



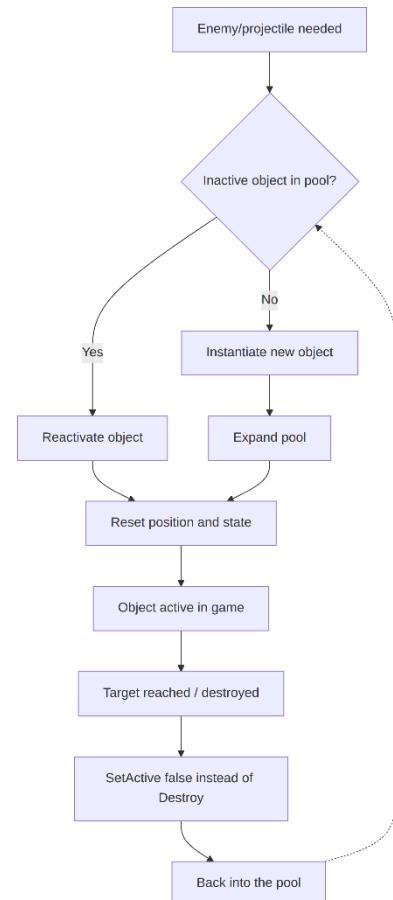
The phase change at 50% HP noticeably raises fire rate and projectile count — the boss gets denser in the second half, not just longer.

Upgrade Selection Flow



The growing threshold (see 5.5) ensures the four upgrades spread out across the whole run instead of clustering early.

Object Pooling Flow



No *Instantiate()*/*Destroy()* during waves - objects are reused, which keeps the frame rate stable with a full screen of activity.

9. Scope

What's actually implemented:

3 enemy types, 1 boss, 1 base weapon, 4 upgrades, endlessly scaling difficulty, Earth and player healthbars, local high score, complete menu/pause/game-over flow.

Deliberately out of scope:

Multiple weapons, multiple bosses, a level/stage structure, online leaderboards, progression between runs.

Guiding principle: finishing and polishing within the defined scope takes priority over additional scope.

10. Possible Extensions

- More enemy types and a second boss — the data-driven setup makes this comparatively cheap to add.
- Meta-progression (permanent unlocks between runs).
- Online/global leaderboard.
- More weapon types selectable at the start of a run.
- Audio (sound effects and music).